PES University, Bengaluru

UE24CS341A Software Engineering | Jackfruit Mini-Project

Software Architecture and Design Specification

Student Attendance Management System (SAMS)

Project: Student Attendance Management System (SAMS)

Version: 1.2

Authors: Kundhan Vudutha (PES1UG24AM327), Tarun J (PES1UG24AM304), Anish (PES1UG24AM348)

Section: AIML 2024, Section F

Date: 29-10-2026

Status: Draft for review

Revision History

Version	Date	Author	Change summary
1.0	30-09-2026	Project team	Initial technical baseline
1.2	29-10-2026	Current team	Revised three-member role allocation and review responsibilities.

Approvals

Role	Name	Signature / Date
Course Coordinator	Prof. Rajeshwari Adrakatti	
Developer	Kundhan Vudutha	
Tester	Tarun J	
QA	Anish	

Table of Contents

Revision History

Approvals

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Audience
- 1.4 Definitions

2. Document Overview

- 2.1 How to use this document
- 2.2 Related Documents

3. Architecture

- 3.1 Goals & Constraints
- 3.2 Stakeholders & Concerns
- 3.3 Component (UML) Diagram
- 3.4 Component Descriptions
- 3.5 Chosen Architecture Pattern and Rationale
- 3.6 Technology Stack & Data Stores
- 3.7 Risks & Mitigations
- 3.8 Traceability to Requirements
- 3.9 Security Architecture

4. Design

- 4.1 Design Overview
- 4.2 UML Sequence Diagrams
- 4.3 API Design
- 4.4 Error Handling, Logging & Monitoring
- 4.5 UX Design
- 4.6 Open Issues & Next Steps

5. Appendices

1. Introduction

1.1 Purpose

This document specifies the software architecture and component-level design of the Student Attendance Management System (SAMS). It turns the requirements in the SAMS SRS v1.0 into a concrete structure: the components the system is made of, the responsibilities and interfaces of each, how they collaborate at runtime, and the technology used to build and deploy them.

1.2 Scope

The design covers every function listed in SRS section 2.2: authentication and role-based access, course/section/timetable setup, per-session attendance marking, the correction workflow, reports and the defaulter list, notifications, administration, and the attendance audit log. Fee management, examinations, biometric hardware and ERP integration are out of scope, as in the SRS.

1.3 Audience

The development team, the course faculty and evaluators, testers writing the System Test Plan, and anyone maintaining SAMS after the course.

1.4 Definitions

Term	Meaning
SAMS	Student Attendance Management System
DRF	Django REST Framework, the library used to build the JSON API
RBAC	Role-Based Access Control (Student, Faculty, Admin)
JWT	JSON Web Token used for stateless API authentication
ORM	Object-Relational Mapping (Django ORM over PostgreSQL)
Session	One scheduled occurrence of a class on a specific date, generated from a timetable slot
OD	On Duty: an excused absence that counts as present
Defaulter	A student whose attendance in a course is below the configured threshold
ADR	Architecture Decision Record
STRIDE	Threat model categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture: goals, stakeholders, the component diagram, each component, the chosen pattern, the technology stack, risks, traceability back to the SRS, and the security architecture. Section 4 describes the design: the class model, sequence diagrams for the main flows, the REST API contract, error handling and logging, UX decisions and open issues. Section 5 holds the glossary, references and tools.

2.2 Related Documents

- SAMS Software Requirements Specification (SRS) v1.0, 05-09-2026
- SAMS Software Test Plan (STP) v1.0, 02-10-2026
- Requirements Traceability Matrix (SRS section 8, extended in section 3.8 of this document)
- Jira project SAM, pes1ug24am327.atlassian.net (product backlog, epics and sprints)

3. Architecture

3.1 Goals & Constraints

Goals

- Correctness of attendance data: a submitted session can never be silently changed (SAMS-F-007, SAMS-SR-005).
- Speed for faculty: a 60-student roll call saved in under one minute, server save under 2 seconds (SAMS-F-006).
- Pages render in 3 seconds or less at the 95th percentile (SAMS-NF-001).
- 99% availability during college hours, 8am to 6pm Monday to Saturday (SAMS-NF-002).
- Modular, testable code with clear implementation, independent testing and QA responsibilities, backed by high branch coverage.

Constraints

- One semester, three-person team: the architecture must stay simple to build, explain and demo.
- Open-source stack only, deployable on lab hardware or a free cloud tier (SRS 2.5).
- Course tooling: Git/GitHub, Jira, Jenkins CI/CD, Docker, pytest, SonarQube.
- Student personal data must never reach unauthenticated users (institutional privacy policy).

3.2 Stakeholders & Concerns

Stakeholder	Primary concerns	How the architecture addresses them
Students	Accurate, current attendance; easy correction requests; privacy	Read-only student endpoints scoped to own records; correction workflow; dashboard refresh within 5 minutes
Faculty	Fast marking between classes; no double entry; simple approvals	Timetable-driven roll list; one-screen bulk save; duplicate-submission guard; pending-request queue
Admin (department)	Setup of courses/timetables/users; threshold config; audit	Admin console over CourseModule and AuthModule; SystemConfig; append-only AuditModule
Course evaluators	Modular design, traceability, CI/CD, security validation	Layered Django apps with clear interfaces; RTM; Jenkins pipeline with tests and SonarQube
Developer	Clear ownership, low coupling, testability	One Django app per module, service functions separate from views, pytest per app

3.3 Component (UML) Diagram

Figure 1 shows the major components and their dependencies. Arrows point from a component to the component it calls. Every service module persists through the Django ORM to a single PostgreSQL database; email is sent asynchronously through a Celery worker.

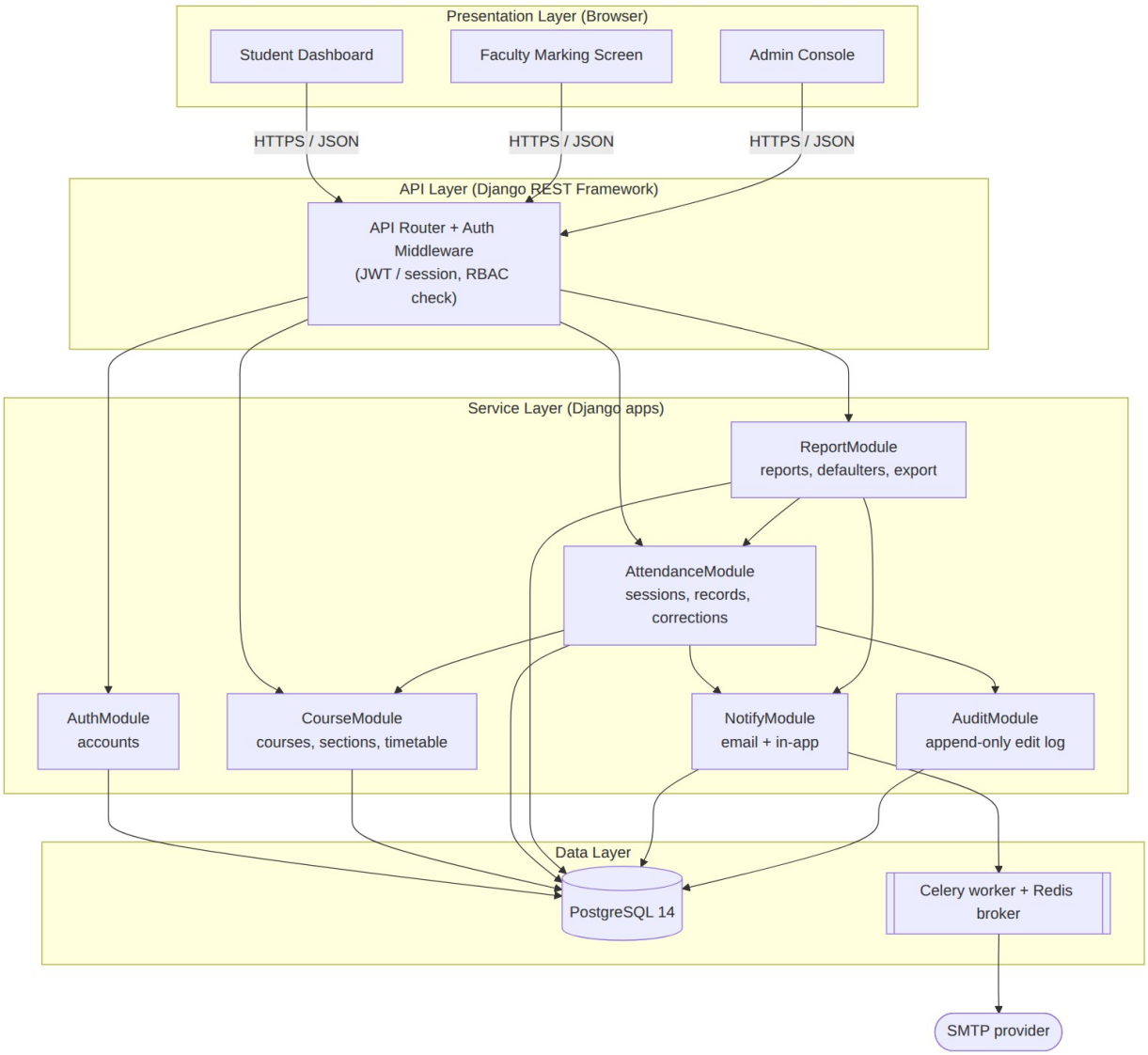


Figure 1: SAMS component diagram (layered)

Figure 2 is the UML class diagram of the domain model that the service layer operates on.

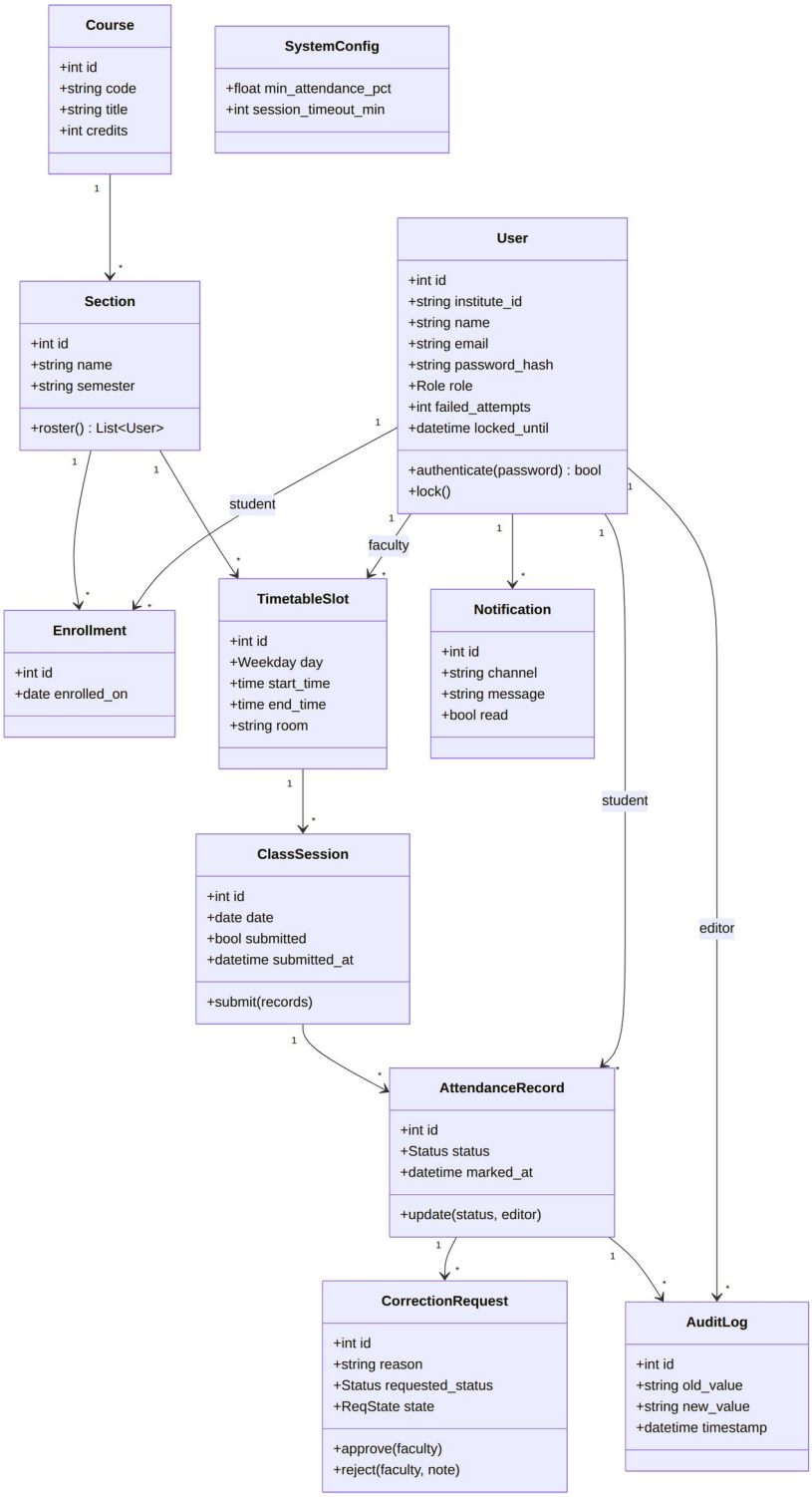


Figure 2: SAMS domain class diagram

3.4 Component Descriptions

Component	Responsibility	Provided interface	Depends on	SRS IDs
Web UI (Student / Faculty / Admin)	Role-specific responsive screens; client-side validation; calls REST API	Browser pages	API Router	SAMS-NF-005, 3.1
API Router + Auth Middleware	URL routing, JWT/session validation, server-side RBAC permission classes on every endpoint, rate limiting on login	/api/* REST endpoints	All service modules	SAMS-F-002, SR-003, SR-004
AuthModule (accounts)	User model, login, lockout after 5 failures, password reset by email link, token issue/refresh/blacklist	authenticate(), lock(), issue_tokens(), reset_password()	PostgreSQL, NotifyModule	SAMS-F-001 to F-004, SR-002

Component	Responsibility	Provided interface	Depends on	SRS IDs
CourseModule (courses)	Courses, sections, enrollments, weekly timetable slots, faculty assignment; generates the roll list for a session	slots_for(faculty, date), roster(section)	PostgreSQL	SAMS-F-005, F-014 to F-016
AttendanceModule (attendance)	Class sessions, bulk marking, duplicate guard, authorised edits, correction requests and decisions, per-student percentages	submit(), edit(), percentage(), create_request(), decide()	CourseModule, AuditModule, NotifyModule	SAMS-F-006 to F-009
ReportModule (reports)	Per-student, per-course and date-range summaries, defaulter list, CSV/PDF export	student_report(), course_report(), defaulters(), export()	AttendanceModule, SystemConfig	SAMS-F-010 to F-013
NotifyModule (notifications)	In-app notifications and queued email for low attendance and pending corrections; nightly defaulter scan	notify(user, msg), nightly_scan()	Celery/Redis, SMTP	SAMS-F-017, F-018
AuditModule (audit)	Append-only log of every attendance create/edit with editor, timestamp, old and new value; read-only query API for Admin	log(), query()	PostgreSQL	SAMS-F-021, SR-005, NF-004
Admin / SystemConfig	User and role management, minimum-attendance threshold, session timeout	Admin endpoints	AuthModule	SAMS-F-019, F-020
PostgreSQL 14	System of record for all entities; constraints enforce uniqueness of (session, student)	SQL via ORM	-	SRS 2.4
Celery worker + Redis	Background email dispatch and scheduled jobs (nightly reports)	Task queue	Redis, SMTP	SRS 3.4

3.5 Chosen Architecture Pattern and Rationale

SAMS uses a **layered architecture** (presentation, API, service, data) implemented as a **modular monolith**: one Django project split into self-contained Django apps, one per component in Figure 1. Views only handle HTTP and permissions; business rules live in service functions inside each app; persistence goes through the ORM.

Architecture Decision Records:

ADR	Decision	Alternatives considered	Rationale
ADR-01	Layered modular monolith in Django	Microservices; plain MVC without service layer	Microservices add network calls, deployment and data consistency work that a three-person one-semester team cannot justify at 2,000 users. A service layer keeps business rules unit-testable without HTTP.
ADR-02	Django + Django REST Framework	Flask, Spring Boot, Node/Express	Built-in auth, admin, ORM and migrations cover a large share of SAMS for free; Python matches the team skill set; SRS 2.4 already fixes Django.
ADR-03	PostgreSQL	MySQL, SQLite	Transactional integrity and unique constraints for the duplicate-marking rule; SQLite is used only for fast unit tests.
ADR-04	JWT (SimpleJWT) with refresh-token blacklist	Server sessions only	Works for the SPA and any future mobile client; blacklist gives real logout (SAMS-SR-003).
ADR-05	Celery + Redis for email and nightly jobs	Synchronous email in request	Keeps attendance save under 2 seconds even if the mail server is slow.
ADR-06	Append-only audit table, no update/delete API	Django simple-history	Satisfies SAMS-SR-005 explicitly and is easy to verify in tests.

3.6 Technology Stack & Data Stores

Layer	Technology
Frontend	Django templates + HTMX and Bootstrap 5 (responsive, 360px and up, WCAG 2.1 AA colours)
API	Python 3.12, Django 5, Django REST Framework, djangorestframework-simplejwt, drf-spectacular (OpenAPI/Swagger)
Data store	PostgreSQL 14+ (production), SQLite (unit tests)

Layer	Technology
Async jobs	Celery 5 with Redis 7 broker, Celery beat for nightly defaulter scan
Reports	Python csv module; WeasyPrint for PDF export
Web server	Gunicorn behind Nginx with TLS 1.2+
Packaging and deploy	Docker, docker-compose
CI/CD	GitHub repository, Jenkins pipeline (lint, pytest with coverage, SonarQube scan, Docker build, deploy)
Testing	pytest, pytest-django, coverage.py, Postman/Newman for API, Selenium for UI flows, Locust for load
Project management	Jira Scrum board (epics, stories, sprints), GitHub pull requests for code review

Figure 3 shows the deployment view.

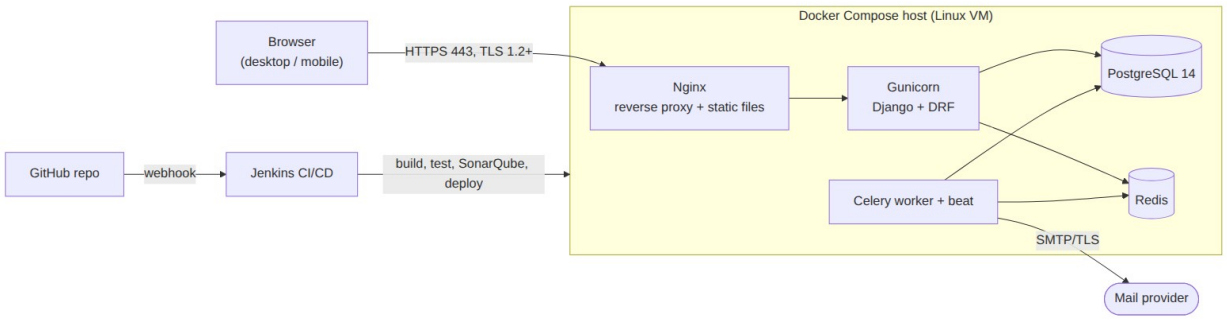


Figure 3: Deployment diagram

Figure 4 is the entity-relationship view of the database.

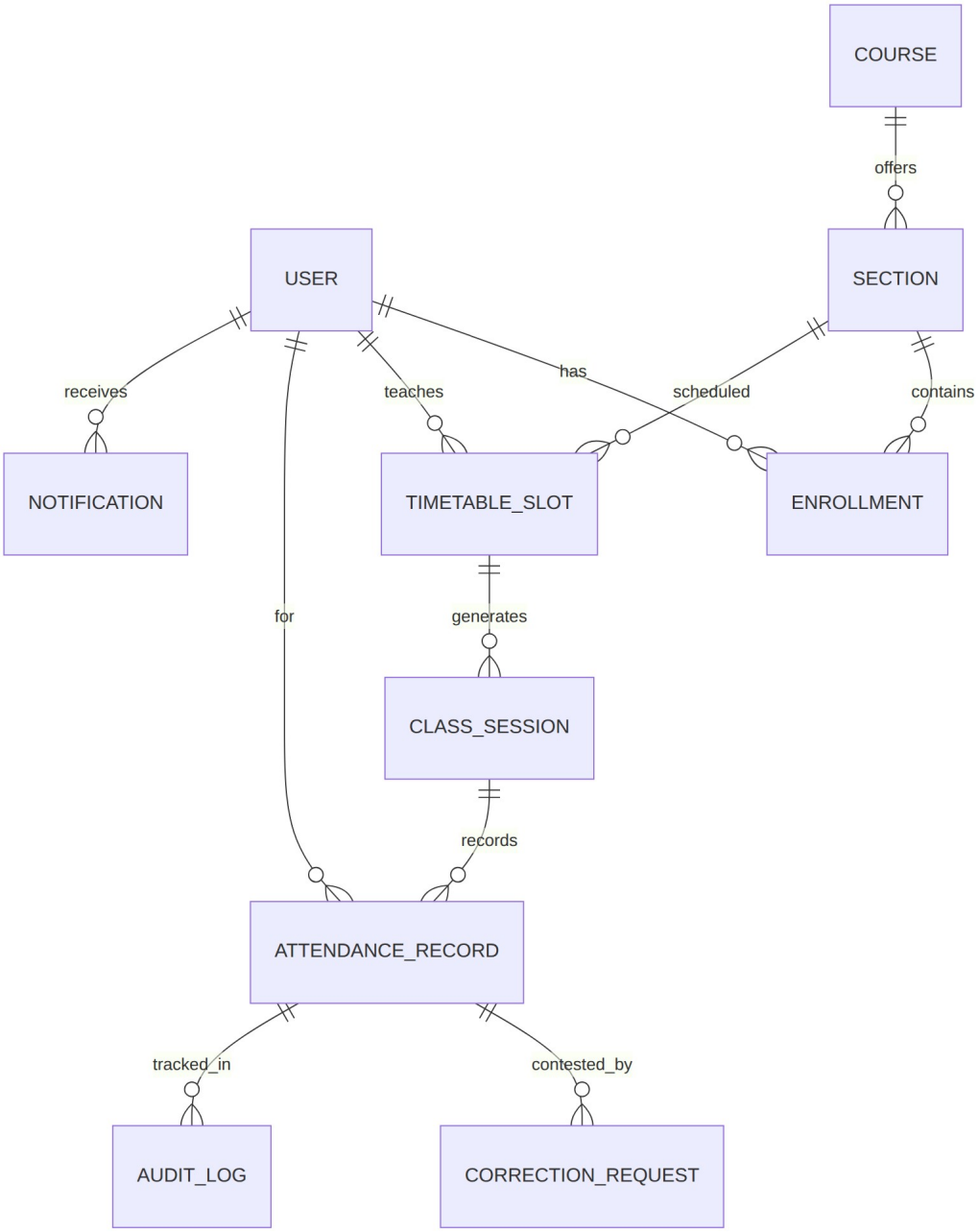


Figure 4: Entity-relationship diagram

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Concurrent submissions for the same session (two tabs, double click)	Duplicate or conflicting records	Unique constraint on (session_id, student_id); submit runs in one DB transaction; 409 Conflict on repeat
Peak load at the start of each period when all faculty mark at once	Slow saves, timeouts	Bulk insert in one query; index on session/date; Locust load test against SAMS-NF-001/NF-006
Mail server outage	Notifications lost or request latency	Celery retry with backoff; in-app notification is written first and does not depend on email
Timetable changes mid-semester	Wrong roll lists	Sessions keep a reference to the slot they were created from; slot edits only affect future sessions
Small team, one member unavailable	Feature slips	Modules have clear interfaces; team members review each other's pull requests; Jira tracks ownership
Leaked or stolen token	Unauthorised access	Short access-token lifetime, refresh blacklist on logout, HTTPS only, HttpOnly cookies

3.8 Traceability to Requirements

Requirement	Architecture element	Design element	Planned code location
SAMS-F-001 Authenticate	AuthModule	Seq. diagram Fig. 7, POST /api/auth/login/	accounts/services.py, accounts/views.py
SAMS-F-002 Role restriction	API Router (RBAC)	Permission classes IsStudent/IsFaculty/IsAdmin	core/permissions.py
SAMS-F-003 Lockout	AuthModule	User.failed_attempts, locked_until	accounts/services.py
SAMS-F-004 Password reset	AuthModule, NotifyModule	Signed one-time token, 30 min expiry	accounts/reset.py
SAMS-F-005 Roll list	CourseModule	slots_for(), roster()	courses/services.py
SAMS-F-006 Mark attendance	AttendanceModule	Seq. diagram Fig. 5, POST /api/sessions/{id}/attendance/	attendance/services.py
SAMS-F-007 No duplicate marking	AttendanceModule, AuditModule	Unique constraint, 409 path, edit endpoint	attendance/models.py
SAMS-F-008 Student view	AttendanceModule	GET /api/students/me/attendance/	attendance/selectors.py
SAMS-F-009 Correction request	AttendanceModule, NotifyModule	Seq. diagram Fig. 6, /api/corrections/	attendance/corrections.py
SAMS-F-010 to F-013 Reports, defaulters	ReportModule	GET /api/reports/*, export	reports/services.py
SAMS-F-014 to F-016 Course & timetable	CourseModule	Admin CRUD endpoints	courses/views.py
SAMS-F-017, F-018 Notifications	NotifyModule	Celery tasks, Notification model	notifications/tasks.py
SAMS-F-019 to F-021 Admin & audit	Admin, AuditModule	Append-only AuditLog, GET /api/audit/	audit/models.py
SAMS-NF-001, NF-006 Performance, scale	Deployment, DB indexes	Bulk insert, pagination	settings, migrations
SAMS-NF-003, SR-001 to SR-005 Security	Security architecture 3.9	TLS, bcrypt, JWT, RBAC, audit	nginx.conf, settings.py

3.9 Security Architecture

Threat model (STRIDE) for SAMS:

Threat	Example in SAMS	Control
Spoofing	Student logs in as faculty using a guessed password	bcrypt-hashed passwords, lockout after 5 failures (F-003), login rate limiting, JWT signature check
Tampering	Student edits their own attendance through a crafted API call	Server-side RBAC on every endpoint (SR-004); only faculty of that section can write records; DB constraints
Repudiation	Faculty denies changing a mark	Append-only AuditLog with editor ID, timestamp, old/new value (SR-005); no update/delete API
Information disclosure	Student reads another student's attendance	Queries scoped to request.user; object-level permission checks; HTTPS only (SR-001); no passwords in logs (SR-002)
Denial of service	Login brute force or report export flood	Nginx and DRF throttling, pagination, async report export
Elevation of privilege	Faculty promotes self to Admin	Role changes only via Admin endpoints; role field read-only in profile serializer; tested in TC-Sec-05

Additional controls: CSRF protection for cookie-based requests, secure/HttpOnly/SameSite cookies, Django security middleware headers (HSTS, X-Frame-Options, CSP), secrets in environment variables, dependency scanning (pip-audit) and SonarQube in the Jenkins pipeline.

4. Design

4.1 Design Overview

Each component in section 3 is a Django app with the same internal layout, so any team member can navigate any module:

```
sams/
  core/            settings, permissions.py (RBAC), exceptions.py, pagination
  accounts/        models.py, services.py, views.py, serializers.py, tests/
  courses/         Course, Section, Enrollment, TimetableSlot
  attendance/      ClassSession, AttendanceRecord, CorrectionRequest
  reports/         report services and CSV/PDF exporters
  notifications/   Notification model, Celery tasks
  audit/           AuditLog (append-only)
  templates/, static/   UI
docker/, Jenkinsfile, docker-compose.yml
```

Design rules: views call services, services call models and other modules' services (never other modules' views); cross-module calls go through the public functions listed in 3.4; every write to attendance goes through AttendanceModule so the audit log cannot be bypassed.

Division of work (one Developer, one Tester and one QA):

Member / role	Accountable responsibilities	Quality / review focus
Kundhan Vudutha PES1UG24AM327 Developer	Implement all application modules: authentication, courses/timetable, attendance/corrections, reports, notifications, administration and audit. Own migrations, CI/CD and fixes.	Functional correctness, secure implementation and reproducible builds.
Tarun J PES1UG24AM304 Tester	Independently execute UC-01 to UC-09 and integration/browser tests. Run performance, security and accessibility checks; report defects and retest fixes.	Actual test results, boundary/error cases and regression coverage.
Anish PES1UG24AM348 QA	Review requirements/design and the test plan, maintain traceability, inspect reports and defect severity, review PRs and assess release readiness.	Quality gates, complete evidence and real peer review.
Team handoff	Developer implements; Tester executes and retests; QA reviews evidence. Faculty UAT remains a separate approval.	Sign-offs stay pending until performed.

4.2 UML Sequence Diagrams

Flow 1: faculty marks attendance for a scheduled session (SAMS-F-005, F-006, F-007).

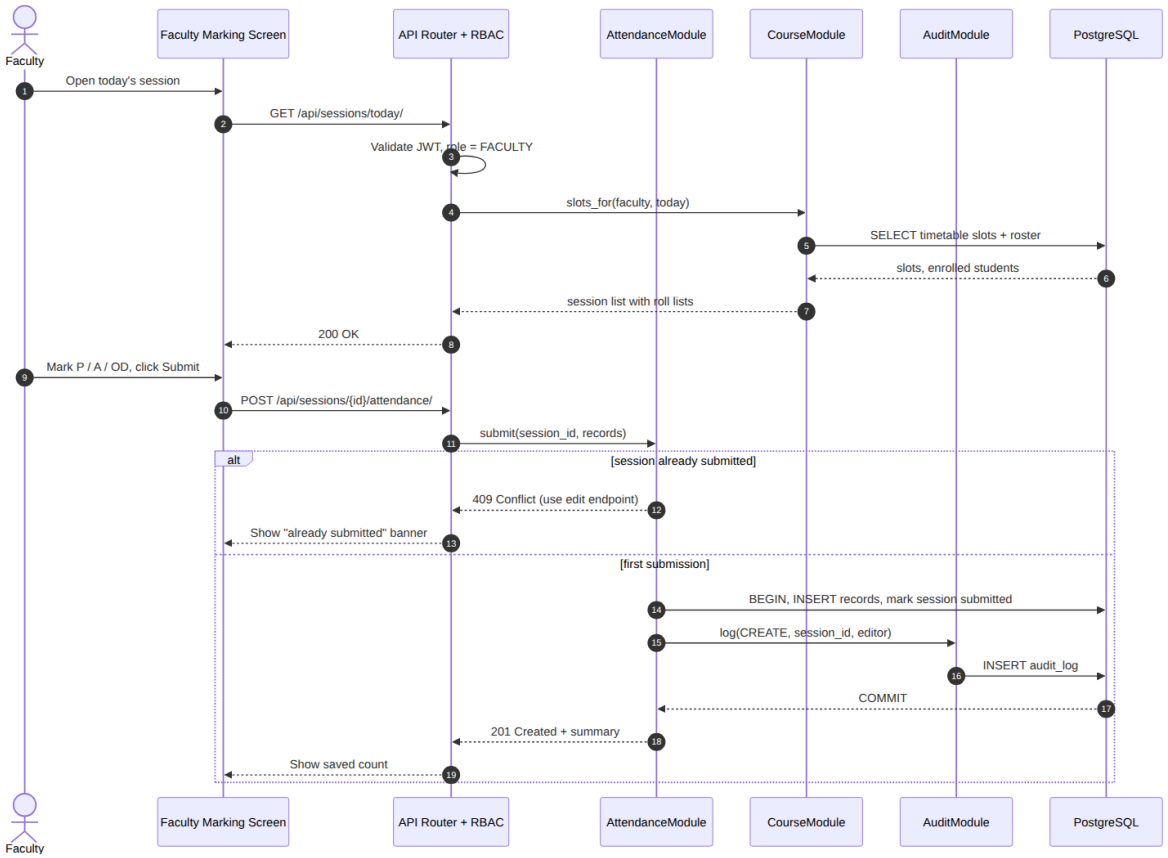


Figure 5: Mark attendance

Flow 2: student raises a correction request and faculty decides it (SAMS-F-009, F-018, SR-005).

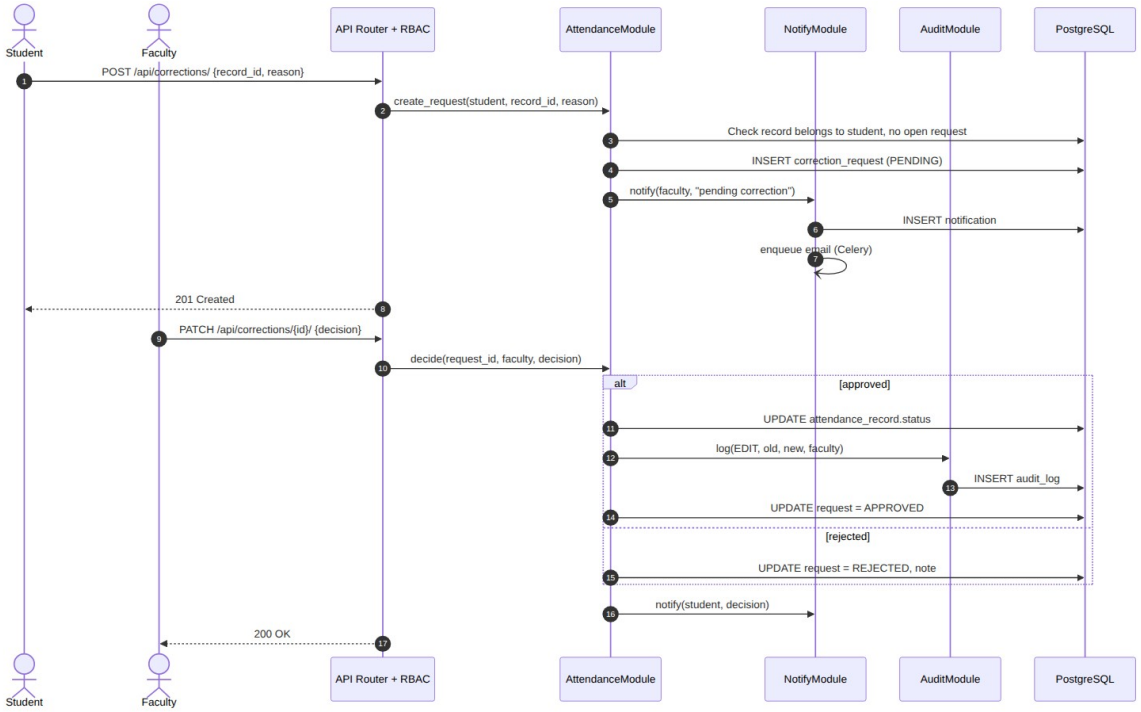


Figure 6: Attendance correction request

Flow 3: login with lockout (SAMS-F-001, F-003).

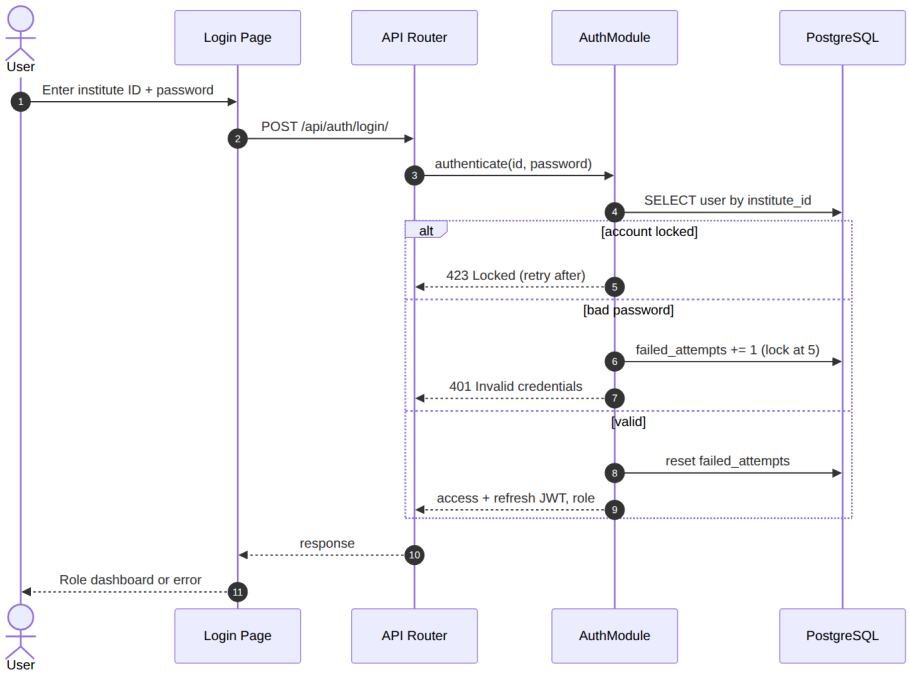


Figure 7: Login and account lockout

4.3 API Design

All endpoints are JSON over HTTPS under /api/, documented with OpenAPI (Swagger UI at /api/docs/). Authentication is a Bearer JWT unless noted. Errors use one shape: {"error": {"code": "...", "message": "...", "details": {...}}}.

4.3.1 AuthModule: Login

Field	Value
Endpoint	/api/auth/login/
Method	POST
Allowed role(s)	Public
Request	{ "institute_id": "PES1UG24AM327", "password": "..." }
Response	200 { "access": "<jwt>", "refresh": "<jwt>", "role": "STUDENT", "name": "..." }
Errors	400 missing fields; 401 INVALID_CREDENTIALS; 423 ACCOUNT_LOCKED { "retry_after_s": 900 }; 429 throttled

4.3.2 AuthModule: Logout

Field	Value
Endpoint	/api/auth/logout/
Method	POST
Allowed role(s)	Any authenticated
Request	{ "refresh": "<jwt>" }
Response	205 Reset Content (refresh token blacklisted)
Errors	401 token invalid or expired

4.3.3 AuthModule: Password reset

Field	Value
Endpoint	/api/auth/password-reset/ and /api/auth/password-reset/confirm/
Method	POST
Allowed role(s)	Public
Request	{ "email": "..." } then { "token": "...", "new_password": "..." }
Response	202 Accepted (same response whether or not the email exists); 200 on confirm
Errors	400 weak password; 410 TOKEN_EXPIRED_OR_USED

4.3.4 CourseModule: Today's sessions

Field	Value
Endpoint	/api/sessions/today/
Method	GET
Allowed role(s)	Faculty
Request	-
Response	200 [{ "session_id": 412, "course": "UE24CS341A", "section": "F", "start": "09:00", "submitted": false, "roster": [{ "student_id": 1, "srn": "...", "name": "..." }] }]
Errors	401; 403 NOT_FACULTY

4.3.5 AttendanceModule: Submit attendance

Field	Value
Endpoint	/api/sessions/{session_id}/attendance/
Method	POST
Allowed role(s)	Faculty assigned to the session
Request	{ "records": [{ "student_id": 1, "status": "P" }, { "student_id": 2, "status": "A" }, { "student_id": 3, "status": "OD" }] }
Response	201 { "session_id": 412, "saved": 60, "present": 54, "absent": 5, "on_duty": 1 }
Errors	400 INVALID_STATUS or student not in roster; 403 NOT_SESSION_FACULTY; 404 session; 409 ALREADY_SUBMITTED

4.3.6 AttendanceModule: Edit a submitted record

Field	Value
Endpoint	/api/records/{record_id}/
Method	PATCH
Allowed role(s)	Faculty of the session, Admin
Request	{ "status": "P", "reason": "Late entry verified" }
Response	200 updated record (audit entry written)
Errors	400 reason missing; 403; 404

4.3.7 AttendanceModule: My attendance

Field	Value
Endpoint	/api/students/me/attendance/?course=UE24CS341A
Method	GET
Allowed role(s)	Student
Request	-
Response	200 [{ "course": "UE24CS341A", "held": 30, "attended": 26, "percentage": 86.67, "below_threshold": false }]
Errors	401; 403

4.3.8 AttendanceModule: Correction requests

Field	Value
Endpoint	/api/corrections/ and /api/corrections/{id}/
Method	POST / PATCH
Allowed role(s)	Student (create), Faculty (decide)
Request	Create: { "record_id": 9001, "requested_status": "OD", "reason": "Hackathon, letter attached" } Decide: { "decision": "APPROVE" | "REJECT", "note": "..." }
Response	201 { "id": 77, "state": "PENDING" }; 200 { "id": 77, "state": "APPROVED" }
Errors	400 reason empty; 403 not own record / not section faculty; 409 OPEN_REQUEST_EXISTS or ALREADY_DECIDED

4.3.9 ReportModule: Defaulter list

Field	Value
Endpoint	/api/reports/defaulters/?course=&section=&threshold=75&format=json|csv|pdf
Method	GET
Allowed role(s)	Faculty (own sections), Admin
Request	-
Response	200 JSON list or file download { srn, name, course, held, attended, percentage }
Errors	400 threshold out of 0-100; 403

4.3.10 AuditModule: Audit log

Field	Value
Endpoint	/api/audit/?record=&editor=&from=&to=
Method	GET
Allowed role(s)	Admin
Request	-
Response	200 paginated [{ "record_id", "editor", "old", "new", "timestamp" }]
Errors	403 for non-admin; 405 for PUT/PATCH/DELETE

4.4 Error Handling, Logging & Monitoring

- A custom DRF exception handler maps every error to the standard error shape with a stable machine-readable code and a user-safe message. Stack traces are never returned to clients (DEBUG off in production).
- Structured JSON logs (request ID, user ID, endpoint, status, latency). Passwords, tokens and reset links are filtered from logs (SAMS-SR-002).
- Security events logged at WARNING: failed logins, lockouts, 403 responses, audit-log access.
- Health endpoint /api/health/ checks DB and Redis for uptime monitoring (SAMS-NF-002).
- Metrics: request latency p95, error rate, attendance saves per minute, Celery queue length, failed emails. Alerts if p95 exceeds 3 s or error rate exceeds 2% for 5 minutes.
- Celery tasks retry 3 times with exponential backoff; permanent failures are written to a dead-letter table shown to Admin.

4.5 UX Design

- Faculty marking screen: all students default to Present; faculty taps only the exceptions, so a 60-student roll call is a few taps. Large touch targets, sticky Submit button, counts shown before submit.
- Student dashboard: one card per course with percentage and a colour plus text label (OK / At risk / Below threshold), never colour alone.
- Admin console: tables with search and filters for users, courses, timetable and audit log; CSV import for bulk enrollment.
- Responsive from 360px width; keyboard navigable; WCAG 2.1 AA contrast on core screens (SAMS-NF-005).
- Errors are shown inline next to the field, in plain language, with the API error code available for support.

4.6 Open Issues & Next Steps

- Decide whether OD requires an uploaded proof document in v1 or only a reason text.
- Confirm the institute's default threshold (75% assumed) and whether it varies per course.
- Choose the SMTP provider for the demo environment.
- Future: QR-code self check-in, ERP integration over the existing REST API, mobile app reusing the JWT API.

5. Appendices

5.1 Glossary

See section 1.4. Additional terms: HTMX (HTML-over-the-wire library for partial page updates), Celery (Python distributed task queue), Gunicorn (Python WSGI HTTP server).

5.2 References

- SAMS SRS v1.0, Kundhan Vudutha and Tarun J, 05-09-2026
- ISO/IEC/IEEE 42010:2022, Architecture description
- OWASP Top 10 (2021) and OWASP ASVS 4.0
- NIST SP 800-160 Vol. 1, Systems Security Engineering
- Django 5 and Django REST Framework documentation

5.3 Tools

Mermaid (UML diagrams), draw.io, Swagger / drf-spectacular (API documentation), Jira (backlog), GitHub (version control and reviews), Jenkins, Docker, SonarQube.